FlightLens: Requirements Specification

DEPI Round 5, Phase 3: Requirements Gathering

Purpose

This document defines what FlightLens must do and how well it must do it: 6 stakeholder groups, 7 user stories, 7 use cases, 11 functional and 8 non-functional requirements. It is the DEPI Phase 3 deliverable (Requirements Gathering) and the checklist the finished system is tested against in weeks 21–22.

The project's goals, architecture, data, plan and risks are in the companion document, *FlightLens: Project Proposal*.

System in one line. FlightLens is a data pipeline built on free, open-source software that turns raw US flight records from BTS into clean, tested, analysis-ready data, served through a Power BI dashboard, a delay-prediction API and a GenAI question-answering assistant.

Stakeholder analysis

Six stakeholder groups use or judge the system; operations managers and analysts are the primary users.

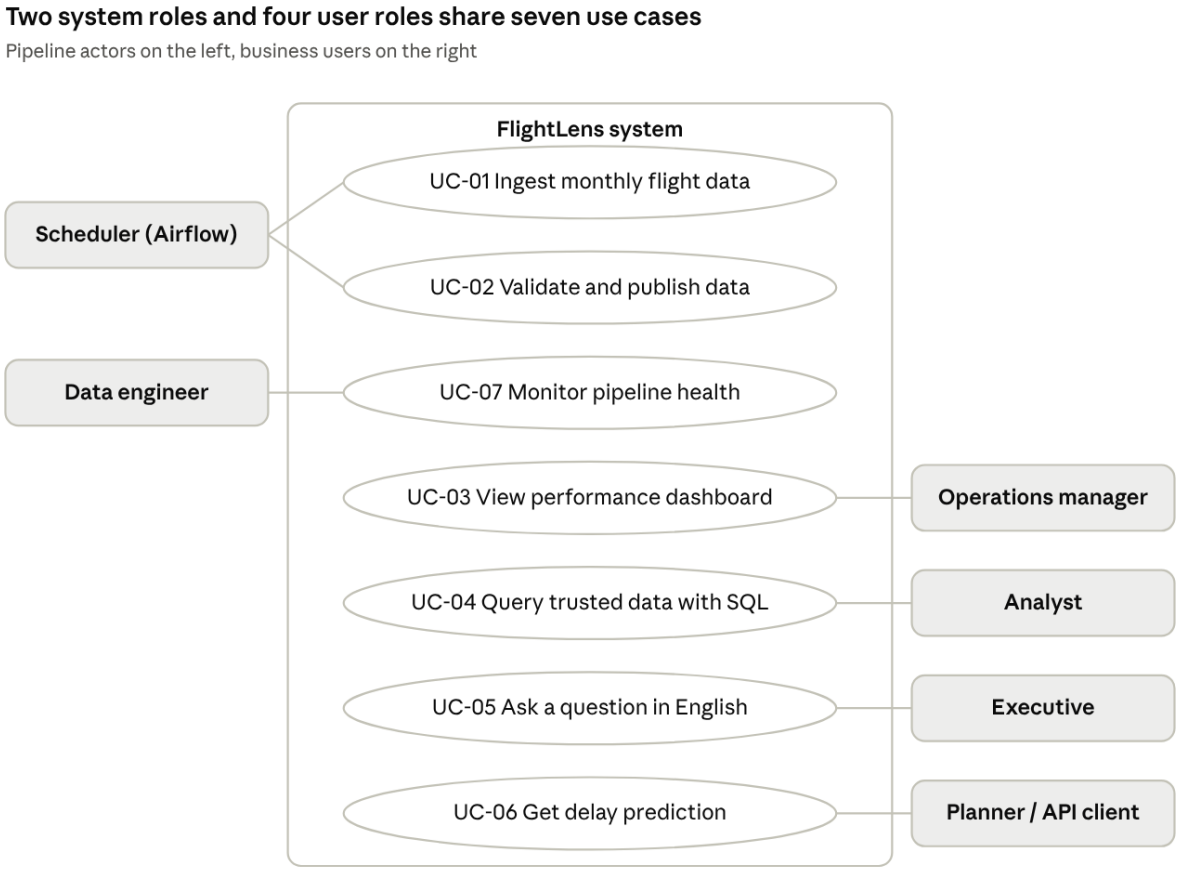
Stakeholder	Need	How FlightLens serves it
Airline operations manager	Know which routes, hours and airports hurt on-time performance	Carriers, Routes and Time patterns dashboard pages
Airport operations manager	Compare the airport with peers and see delay causes	Airports and Delay causes pages, filtered by airport
Business / data analyst	Clean, documented data to query without re-cleaning	Gold star schema, data dictionary, SQL access
Non-technical executive	Quick answers without writing SQL	GenAI assistant
Data engineering team (us)	Reliable, re-runnable loads and visible failures	Orchestrated pipeline, quality checks, alerts, data-health page
Lecturer / evaluator	Evidence of every course module and of quality	Course mapping, tests, documentation, live demo

User stories

ID	As a…	I want…	So that…	Use case
US-01	Airline operations manager	to see on-time rate by route and departure hour	I can adjust schedules on the worst routes	UC-03
US-02	Airport manager	to rank airports by delay and cancellation rate for a chosen	I can compare my airport with peers	UC-03

		month		
US-03	Analyst	one trusted fact table with documented columns	my numbers match everyone else's	UC-04
US-04	Analyst	delay minutes split by cause	I can tell controllable delays (carrier) from uncontrollable ones (weather, air traffic)	UC-03, UC-04
US-05	Executive	to ask "Which carrier had the most cancellations in December?" in plain English	I get the answer, and the SQL behind it, without writing code	UC-05
US-06	Planner	the delay probability of a scheduled flight	I can flag high-risk flights in advance	UC-06
US-07	Data engineer	each monthly load to run automatically and alert me on failure	bad data never reaches Gold	UC-01, UC-02, UC-07

Use case diagram



Use case diagram · 6 actors, 7 use cases

The pipeline use cases (UC-01, UC-02, UC-07) run without a business user; every business use case reads only the Gold layer they produce.

Use case descriptions

ID	Use case	Actor	Trigger	Main flow	Alternative / failure flow
UC-01	Ingest monthly flight data	Scheduler (Airflow); data engineer for backfill	1st of the month, or manual backfill	Download the BTS file for the month → store unchanged in Bronze with load time and file name	Download fails → retry 3 times → mark run failed and alert
UC-02	Validate and publish data	Scheduler	UC-01 finished	Clean to Silver → run quality checks → build Gold tables → refresh warehouse views	Critical check fails → move rows to quarantine, stop the run, alert
UC-03	View performance dashboard	Airline / airport operations manager	Opens Power BI report	Pick date range, carrier, airport → see KPIs, maps and rankings	No data for the filter → page shows "no flights" instead of zeros
UC-04	Query trusted data with SQL	Analyst	Needs a custom analysis	Connect to warehouse with read-only user → query Gold views using the data dictionary	Query on a raw layer → access denied (read access to Gold only)
UC-05	Ask a question in plain English	Executive	Types a question in the assistant	Assistant sends question + schema to the LLM (Ollama or Gemini) → gets SQL → runs it read-only → shows answer, table and SQL	SQL invalid or not SELECT → not run; assistant asks to rephrase
UC-06	Get delay prediction	Planner (via app or API call)	Sends carrier, route, date and hour	API builds features → model returns delay probability and on-time / delayed label	Unknown airport or carrier → 400 error with a clear message
UC-07	Monitor pipeline health	Data engineer	Alert email, or daily check	Open data-health page → see last load, row counts, check results → fix and re-run the month	Re-run replaces that month only (no duplicates)

Functional requirements

ID	Requirement	Use case	Priority
FR-01	Download BTS on-time data for a given month automatically and on a monthly schedule, plus a backfill of July–December 2025	UC-01	Must
FR-02	Store every raw file unchanged in the Bronze layer with load metadata	UC-01	Must
FR-03	Clean, type and standardise records into Silver (times, nulls, flags, lookups)	UC-02	Must
FR-04	Run data-quality checks on each load and quarantine failing rows	UC-02	Must
FR-05	Build the Gold star schema and daily KPI aggregates	UC-02	Must
FR-06	Join hourly weather to flights by airport and hour	UC-02	Could
FR-07	Expose Gold tables through a SQL endpoint with read-only views	UC-04	Must
FR-08	Provide a Power BI dashboard answering Q1–Q7 with date, carrier, airport and state filters	UC-03	Must
FR-09	Train, evaluate and register a delay-prediction model; serve predictions through a REST API	UC-06	Should
FR-10	Answer plain-English questions by generating and running read-only SQL, showing the SQL used	UC-05	Should
FR-11	Show pipeline status and data-quality results on a data-health page	UC-07	Must

Must = needed to pass; Should = planned, dropped only if the schedule slips; Could = stretch, only if time remains.

Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Monthly incremental load under 30 minutes; dashboard pages load in under 5 seconds; prediction API under 500 ms; assistant answer under 10 seconds
NFR-02	Scalability	Handles July–December 2025 (3,554,943 rows) and grows month by month without redesign (partitioning by year and month)
NFR-03	Reliability	Re-running a month gives the same result (idempotent); failed runs retry and alert; 95% of scheduled runs succeed
NFR-04	Data quality	100% of critical checks pass before data reaches Gold
NFR-05	Security	Secrets in an .env file kept out of Git; role-based access; assistant uses a read-only SQL user
NFR-06	Usability	Dashboard usable without training; assistant answers in plain

		language
NFR-07	Maintainability	Modular, tested Python code; one-command Docker Compose setup; README and data dictionary
NFR-08	Cost	Runs on team laptops with free, open-source software; no paid cloud services